

任务一项目测试报告

使用 TINY 的正则表达式进行测试，经过对输入格式适配性处理后，对应的输入内容如下所示：

```
digit=[0-9]
letter=[a-zA-Z]
_key100S=if|then|else|end|repeat|until|read|write
_special200S=\+\\-|\\*\\|/|%|<|<>|<=|>|=|:|=\\(\\)\\
_specialName300S=PLUS|MINUS|MULTIPLY|DIVIDE|MOD|LT|NE|LTEQ|
RTEQ|RT|EQ|SEMI|ASSIGN|LPAN|RPAN
_ID400=letter(letter|digit)*
_NUMBER500=digitdigit*
_annotation600={~}
```

使用的 simple.tny 测试文件内容如下：

```
{ 这是一个简单的 Tiny 语言示例程序 }

{ 主程序开始 }

read n ;

sum := 0 ;

i := 1 ;


{ 输出结果 }

write sum ;
```

```
{ 判断 n 是否为偶数 }
```

```
if  $n \% 2 = 0$  then
```

```
    write 0
```

```
else
```

```
    write 1
```

```
end ;
```

```
{ 另一个循环示例 }
```

```
repeat
```

```
     $n := n - 1 ;$ 
```

```
    write n
```

```
until  $n = 0$ 
```

接下来，运行启动任务一的项目系统后，输入或从文件导入上面准备的 TINY 语言的正则表达式，点击“开始分析”按钮进行分析，之后再依次点击“NFA 生成”、“DFA 生成”、“DFA 最小化”、“词法分析程序生成”对各项结果进行查看：

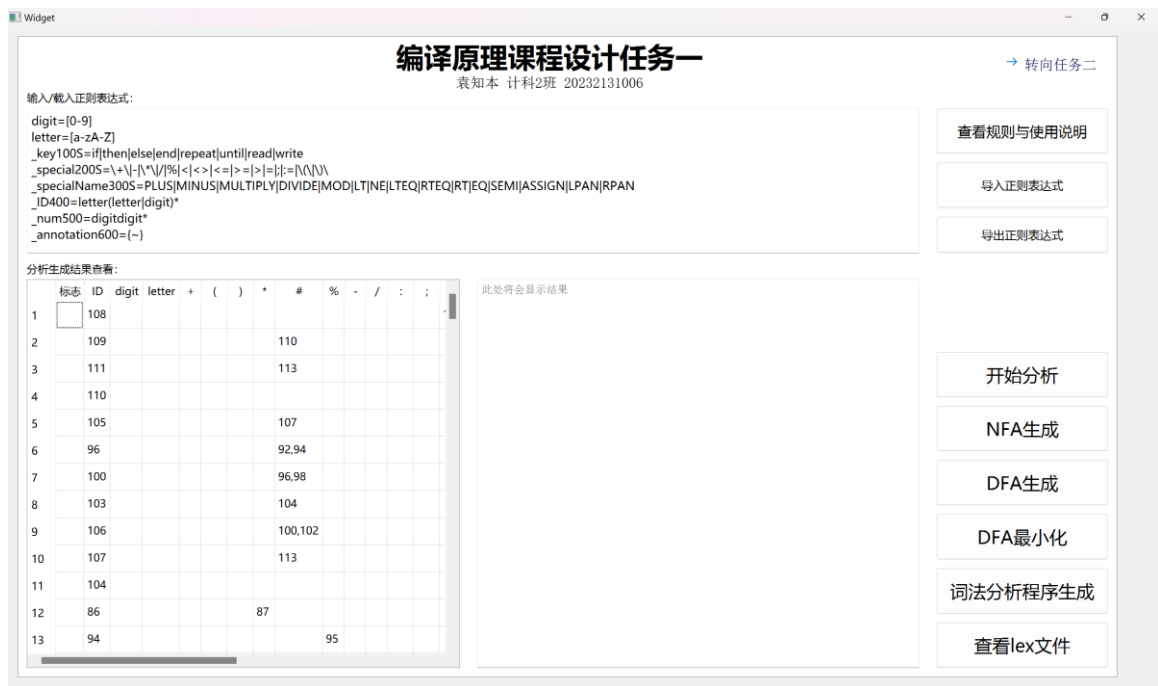


图 1 构建 NFA 测试

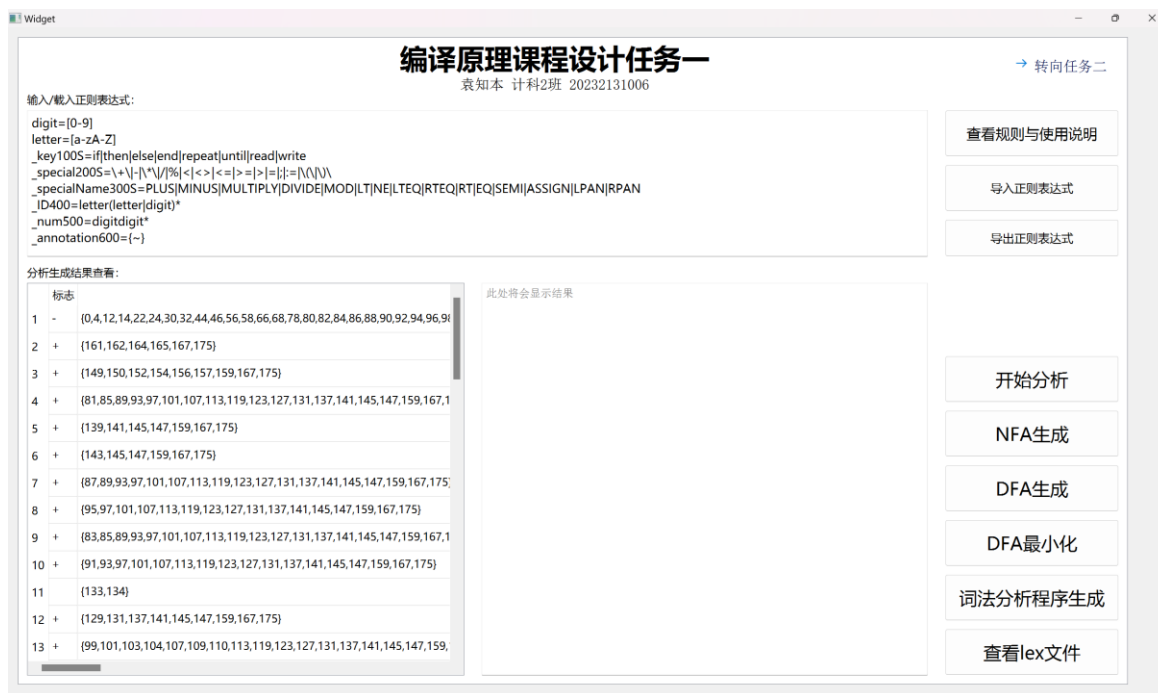


图 2 NFA 转 DFA 测试

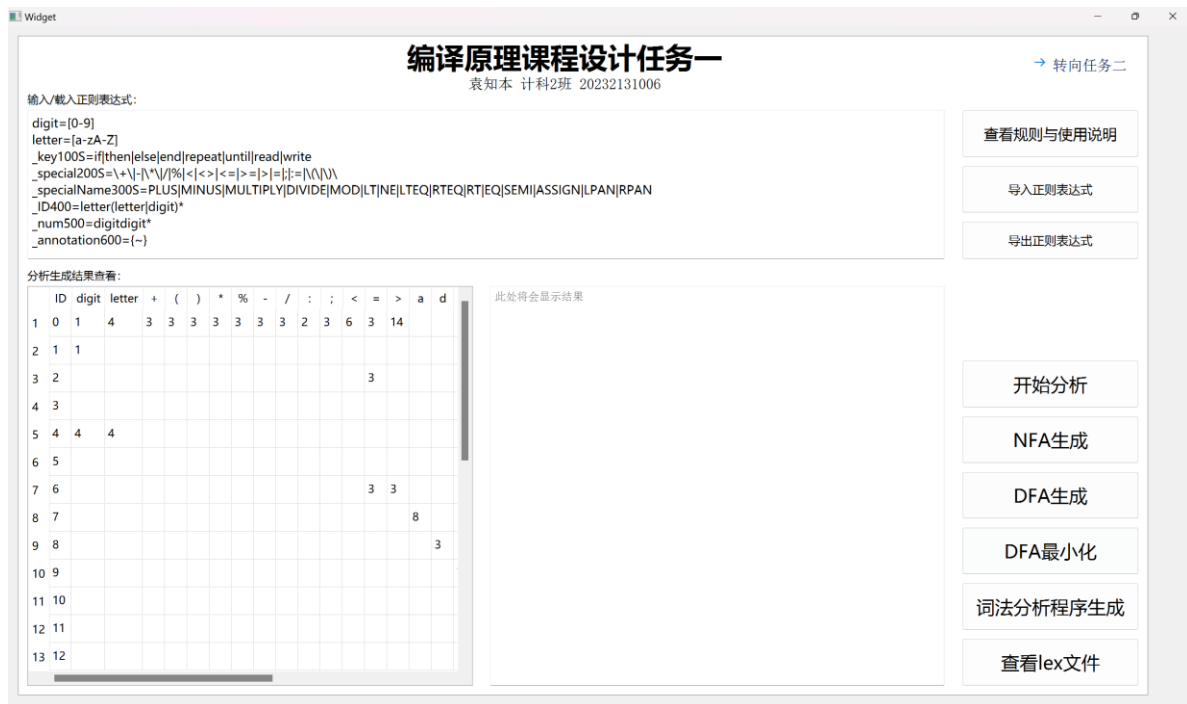


图 3 DFA 最小化测试

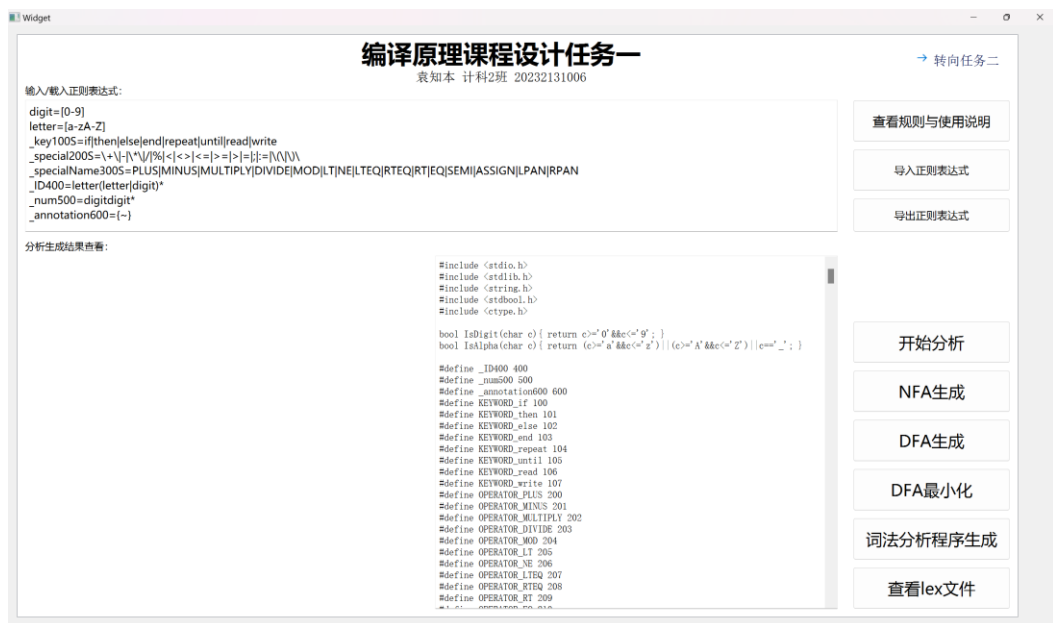


图 4 生成词法分析程序测试

_lexer.c 文件中保存的即为生成的词法分析程序，通过指令【gcc -std=c99

【_lexer.c -o lexer】编译保存下来的_lexer.c 文件后，生成了可执行程序 _lexer.exe，此时将测试用的 sample.tny 放到同一文件夹下，运行可执行程序，即可得对应的 output.lex 编码程序文件，再通过任务一项目系统的“查看 lex 文件”按钮实现查看 output.lex 文件内容：



图 6 lex 文件查看测试

测试结果如下：

经过测试，TINY 语言的正则表达式输入后测试所有的功能均通过，系统能实现对 TINY 正则表达式的分析并生成 NFA、DFA 图（以表格形式展示），进行 DFA 图最小化并展示（表格形式），生成对应的语法分析程序，并能对 lex 文件进行查看，全部测试都能通过。